Proceedings of the 2011 Industrial Engineering Research Conference
T. Doolen and E. Van Aken, eds.

Simultaneous Task Scheduling and Resource Binding for Digital Print Automation*

Mukesh Agrawal, Krishnendu Chakrabarty
Duke University
Durham, NC

Jun Zeng, I-Jong Lin, Gary Dispoto
Hewlett-Packard Laboratories, Hewlett-Packard Co.
Palo Alto, CA

Abstract

On-demand digital printing is a form of personalized manufacturing service. It provides unique opportunities to automate the printing process, enhance productivity, and better utilize resources (equipment, servers, IT infrastructure, etc.). In this work, we demonstrate how concepts from electronic design automation can be adapted for an important optimization problem in digital printing, viz., simultaneous mapping of component tasks of a print job to time steps (scheduling), selection of resources for these tasks, and mapping of tasks to resources (binding). We model print jobs, the relationships between them, and dependencies between tasks within a job, in terms of sequencing graphs. This formal representation is then used for scheduling and resource binding. The optimization objective is to enable just-in-time manufacturing, that is, to minimize the slack time for job orders, i.e., the duration between the delivery deadline and the completion time of the order. The optimization is carried out under the constraint that no job order misses its delivery deadline. The proposed approach uses genetic algorithms (GA) to systematically search the space of feasible solutions. The fitness function of the GA is carefully crafted to match the optimization objective. The optimization technique is evaluated using print orders from a commercial print service provider and compared to baseline methods commonly implemented at the industrial settings.

Keywords

Digital printing, electronic design automation (EDA), genetic algorithms, on-demand digital print services, queue management optimization, scheduling.

1. Introduction

Commercial print has annual retail sales over US$700B [1]. Key to the commercial print value-creation chain is the service engagement and fulfillment between content suppliers and print service providers (PSP). The content suppliers are the service clients; they request print services and supply content for print. They range from traditional publishing agencies for newspapers, books and periodicals to web-based digital media companies such as *www.snapfish.com*. The PSP leases out its manufacturing capability to the content suppliers as a form of utility service in exchange for payment [1]. "Everything that can become digital, will become digital".† Leading the digitization of commercial print is the on-demand digital print services. Today it commands an impressive 8% annual growth [1]. As a form of on-demand manufacturing, digital print deals with high-frequency and high-volatility service demands. Because the heterogeneous products it produces are physical artifacts and involve manual procedures and heterogeneous and legacy equipments, there exist high process variations. The orders (e.g., request for prints) usually have short fulfillment cycle time and late delivery incurs extra shipping cost. The complexity and demand for near-real-time decision pushes the factory design and operations management of digital print services from a form of "tribal art" towards a paradigm of data driven science where the manufacturing process, resource utilization and workforce deployment must be optimized to maximize the business objective.

*This research was supported in part by a grant from HP Labs Open Innovation Research Program. Zeng, Lin and Dispoto would like to thank our collaborators at Reischling Press, Inc., in particular, Susan Jackson, Robert Mitchell and Mark Gustafson.

†Quoted Benny Landa, founder of HP Indigo and inventor-entrepreneur who introduced the first successful color digital press.

The authors in [1] have defined a digital print ecosystem of various entities having different fulfillment needs and resource requirements. Customers request for prints either by physically visiting the PSP or by electronically submitting them to content suppliers using the internet. A PSP receives the orders that are to be fulfilled within a certain deadline. An order is a contract for print service given to a PSP; it includes an order identifier, due date, number of copies, price and payment information, customer information, inks and presses, finishing and shipping methods. Several printing processes such as Raster Imaging Process (RIP), cover printing, page printing, folding, collating, binding, packaging and other related processing stages must be scheduled and finished before the completed order can be shipped to customers. The workflow is highly variable, the job mix is dynamic, and the available resources have differing capabilities. Therefore, simultaneous scheduling and resource binding are necessary to achieve streamlined and efficient production quality.

When multiple orders arrive on a job floor and compete for resources, urgent orders must be given higher priorities and must be scheduled before the less urgent orders. The PSPs are accountable for contractual commitment made to their customers. The runtime policy adopted on the floor guides the schedule created along with resource binding in an attempt to optimize a given objective function. This paper discusses one such objective function and a genetic algorithm-based heuristic for optimization. Before going into the details of the heuristic, we describe the problem formulation in the next section.

2. Problem Description and Formulation

Fulfillment of an order is composed of a sequence of tasks that are serviced by different types of resources at each stage. Each resource can be described by its speed and the function that the resource performs. For example, a color printer's printing rate of 1 *second per page* or a binder's mean service time of 50 *seconds* describes the speed of the corresponding resources. Given a set of n orders with deadline d_i for order i and a description of all the resources available, we attempt to create a schedule of all tasks belonging to all orders to obtain the completion time c_i. The objective is to minimize the sum $\sum_{i=1}^{n}(s_i = d_i - c_i)$, where $d_i \geq c_i$. The difference s_i between the deadline and the completion time for order i is called the slack time. However, the minimization of the above objective function can lead to the missing of deadlines for some orders, i.e., $c_i > d_i$ or $s_i < 0$. Therefore, we adopt a modified objective function, whereby we minimize $\sum_{i=1}^{n} \alpha_i s_i$. Note that $\alpha_i = 1$ for $s_i \geq 0$ and $\alpha_i = M$ (a large negative constant) for $s_i < 0$. M signifies the penalty incurred by a late order, for instance, extra cost to elevate the shipping methods, or the negative impact to future demand generation.

Our optimization problem is different from a general job-shop scheduling problem since we do not minimize the sum of tardiness of the jobs. This problem formulation is motivated by two reasons: (i) we do not want the factory floor to be idle for extended periods of time if the orders have relatively loose deadlines; (ii) since we are targeting real-time print operations, we are penalizing those schedules that violate the order deadlines. With this objective, we aim at creating tight schedules for critical orders and a relaxed schedule for orders having loose deadlines. We are only considering non-preemptive scheduling (scheduling without interrupting any task while it is being processed) without any idle time inserted in the schedule.

The payload of an order may be composed of several book titles. We refer to each book title as a product. An order is an entity composed of several products. A product's information contains the digital file that will be used for preprocessing and printing, quantity (number of copies) and a fulfillment path ID. A *fulfillment path* defines, for a particular type of product, an ordered sequence of tasks or processing steps that completes the creation of a product of this type. For example, printing a poster requires a different set of resources and a different fulfillment path from that of soft-cover book printing product. Within the print factory, there are diverse set of resources (e.g., machines). The procedure of resource binding addresses the assignment of a task to a suitable machine chosen according to the desired operations objective. A product may be composed of more than one part. In case of multi-part products, the corresponding fulfillment path starts with parallel fabrication of the parts (e.g., printing the book cover and printing the book block) followed by part assembly into a product. In a similar manner, several products merge to form a complete order. Based on its scope, a processing stage can be categorized into an order-level, a part-level, or a product-level stage.

Figure 1 shows an example of two simplified fulfillment paths. These paths are used in our test cases for evaluat-

ing the proposed optimization method. The order-level stages *Serializer* and *Packager* are common to all products. The fulfillment path A has two parts; one part for printing a cover and second part to print a book block. The two parts join together at the *Binding* stage to mark the end of part processing. The two parts can be processed in parallel and are independent of each other for their completion. However, *Binding* requires both parts to be completed before the *Binding* task of a product begins. The fulfillment path B processes a single-part product and has no such constraint on the *Binding* stage.

Several penalty-based optimization problems are discussed in the review paper by Baker and Scudder [3]. Even for a single machine, sequencing with earliness and tardiness penalties (even for a common due date) is an NP-complete problem [3]. If we consider only one processing stage, our problem is equivalent to the single-machine problem with deadline penalty. Therefore, the general problem that we are attempting to solve is computationally at least as hard as known NP-complete problems; it cannot be solved optimally using any deterministic polynomial-time algorithm. Heuristic methods are therefore needed for practical problem instances.

3. Related Prior Work

The heterogeneity of arriving orders and varying customer requirements present a major challenge in achieving high productivity in digital printing [1, 2]. A simulation platform for commercial digital print operations has been developed recently to understand several lean manufacturing rules [1, 4]. This platform can be used to evaluate optimization methods and decisions before their adoption on the factory floor. Document production was first perceived as a manufacturing process in [2]. Operations research-based productivity solutions were developed and deployed at several sites to achieve unprecedented productivity and cost improvements for print shops [2]. However a limitation of [2] is that the interrelated scheduling and resource-binding problems are tackled independently, as opposed to the co-optimization approach that we propose in this work.

The digital print automation problem has striking similarities with electronic design automation (EDA), especially high-level synthesis [5]. The sequencing problem in high-level synthesis is modeled using a sequencing graph that captures dependencies between operations. The problems of scheduling and resource binding are solved using heuristic methods that are scalable for large designs. The minimization of chip area, clock cycle time, power consumption are typical optimization goals that are targeted in EDA. The digital-print optimization problem can be modeled using high-level synthesis; state-of-the-art EDA algorithms can be leveraged to solve this problem. On a similar note, recent work in the area of digital microfluidics has also highlighted similarities between EDA and reconfigurable biochip design [6]. Classical high-level synthesis have been adopted for the design and optimization of digital microfluidic biochips. In particular, a heuristic method based on genetic algorithms (GA) was adopted in [6] to solve the biochip synthesis and optimization problem. Motivated by the similarity between reconfigurable biochip design and digital print optimization, we adopt a GA-based heuristic in this work.

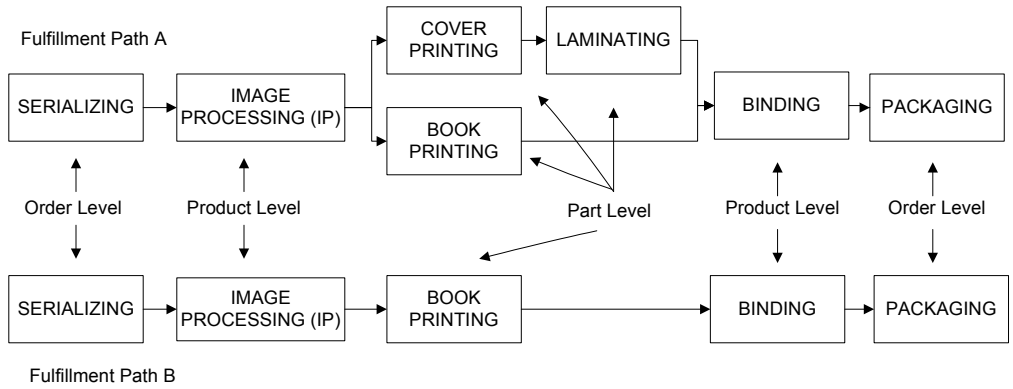


Figure 1: An illustration of fulfillment paths.

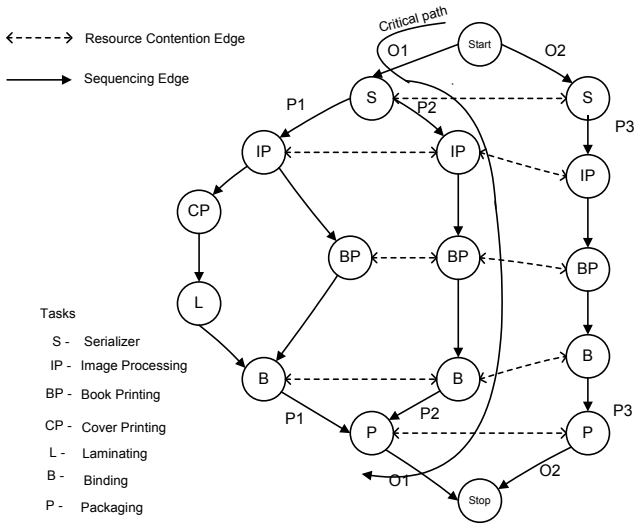


Figure 2: An example of a task sequencing graph.

4. Heuristic Approach

GAs have been used to effectively solve optimization and search problems in various domains [7]. Along with minimizing the sum of slack times, we aim to optimize resource binding without violating the due-date commitment. Therefore, the problem at hand is a constrained optimization problem for which GAs have been widely used. Moreover, a GA-based approach allows to escape local minima by maintaining a pool of possible solutions and by evolving the population in each generation using evolution techniques. Before describing the details of the heuristic method, we discuss the procedure used to construct a task sequencing graph from the set of orders.

4.1 Task Sequencing Graph (TSG)

An abstract model of print operations at the system level can be developed in terms of operations and the dependencies between them. We use the sequencing graph model from high-level synthesis terminology [5]. In a sequencing graph, each print operation or a task is modeled as an operation node. First, we identify all the tasks of an order, create an operation node for each one of them and then define the dependencies between them. This can be better explained with the help of a simple example. In Table 1, we consider two orders O1 and O2. Note that O1 has products P1 and P2 while O2 has a single product P3. While P1 is to be processed using the fulfillment path A, P2 and P3 are to be processed using the path B. There are different types of sizes used in this example: order size, product size and part size representing the size of tasks at the corresponding levels. Product P1 has two part sizes because path A has two parts. For a given order, we create one operation node for each order-level processing stage. Similarly, we create one node for each product-level stage and part-level stage for each product and part, respectively. The final task sequencing graph is shown in Figure 2. The *start* and *stop* nodes are no-operation nodes used only for the purpose of system-level integration of all tasks. The *sequencing* edges mark the sequence in which operations are to be carried out, thereby indicating the precedence constraint between them. The *resource contention* edges connect similar tasks that are contending for the resources from the same resource set. The set of nodes interconnected by these type of edges is a *task set*. For an operation node v in the TSG, we use $childs(v)$ and $parents(v)$ to denote the set of child and

Table 1: An example of orders.

Order	Order Size	Product	Product Size	Fulfillment Path	Part Sizes
O1	15	P1	10	A	2, 50
		P2	12	B	60
O2	50	P3	20	B	80

Table 2: An example of a resource library.

Resource Set	Quantity	Speed(mean)
Serializers	5	35
Image Processors	2	20
Book Printers	10	1 sec/sheet
Cover Printers	1	5
Laminators	2	50
Binders	2	42
Packagers	2	23

```
1: for each task v_i in the system do
2:     Compute ω(v_i), an estimate of worst-case execution delay
3:     Initialize δ(v_i), deadline of v_i
4: end for
5: Generate Initial Population P(t) of chromosomes
6: while termination criterion not reached do
7:     for each chromosome of P(t) do
8:         Create Schedule using procedure in Figure 4.
9:         Determine fitness value, f
10:    end for
11:    Reproduction P(t) → P(t+1);
12:    CrossOver P(t) → P(t+1);
13:    Mutation P(t) → P(t+1);
14:    P(t) ← P(t+1);
15: end while
```

Figure 3: Heuristic Procedure based on GA.

```
1: for each task set T do
2:     for each task v_i ∈ T do
3:         estimate σ(v_i), slack Time of v_i,
4:         Assign a Resource
5:     end for
6:     Sort T based on σ(v_i) to determine p(v_i), ∀v_i ∈ T
7:     Compute effective priority ρ(v_i), ∀v_i ∈ T
8:     Sort T based on ρ(v_i)
9:     Schedule all tasks in T
10: end for
```

Figure 4: Procedure to create a schedule.

parent nodes of v, respectively.

4.2 Resource Set and Task Set

Based on the capabilities of each resource, the entire resource pool is partitioned into different *resource sets*. Resource sets for the fulfillment path example in Figure 1 are shown in Table 2. Resources within the same resource set can process similar tasks; these tasks compete with each other for resource allocation. We define a *task set* as a set consisting of similar tasks competing with one another for the resources belonging to the same resource set. All tasks in the system can be partitioned into multiple task sets. In Figure 2, we have 7 task sets; there is a one-to-one correspondence between task sets and resource sets.

4.3 Chromosome Representation and Decoding

GA manipulates a set of candidates, called population, in each generation. The candidates are called chromosomes. It is very important to devise a good encoding mechanism to ensure fast and unambiguous decoding into a valid schedule. We use the *random keys* representation technique [8] for our heuristic. With this technique, each chromosome can be easily decoded into a schedule. The use of *random keys* also ensures that the offsprings are candidates. Each chromosome in the population is encoded as a vector of random keys. If the total number of operation nodes in the TSG is m, each chromosome can be described as a set: $Chromosome = \{gene(1), gene(2), ..., gene(2m)\}$. Each gene is a random number (fraction) between 0 and 1. The first m genes are used for encoding the priority of the tasks and the last m genes are used for allocating a resource to these tasks. The tasks are numbered from 1 to m in the order they are visited when the TSG is traversed in a breadth-first style starting from the start node. If a task v_i can be processed using a resource from a resource set R, the resource assigned to v_i is $\lceil gene(m+i) \times |R| \rceil$ where $1 \leq i \leq m$.

Before a task is scheduled, its priority is determined dynamically, rather than inherited statically from the parent tasks that have already been scheduled. The heterogeneity of order composition due to varying print requirements is a motivation for adopting this approach. It may not be sensible to assign the same priority to a 500-page product and a 50-page product just because they belong to the same order. Assigning a numeric value to a task's priority entails deducing the order deadline into deadline for each task and estimating the completion time of each task. The difference between the estimated deadline and the estimated completion time of a task provides an estimate of the slack time per task, which serves as basis to decide individual priorities of the tasks. Figure 3 outlines the heuristic procedure based on GA. We define $\omega(v_i)$ as the worst-case estimate of the delay in completing task v_i when all other competing tasks are also ready for processing. The parameter $\omega(v_i)$ is computed as follows:

- If v_i is in *task set* T and is serviced by *resource set* R, compute $g = \lceil |T|/|R| \rceil$. Create g groups and distribute tasks from T into these groups.
- Place v_i in the group g and assign the slowest resource r_s from R to v_i. From the remaining tasks in T, place the largest task in group 1, second largest in group 2 and so on until group $g-1$. Assign the resource r_s to all of them. The parameter $\omega(v_i)$ is computed as:

$$\omega(v_i) = t(r_s, v_i) + \sum_{k=1}^{g-1} t(r_s, \tau_k(T - \{v_i\})) \tag{1}$$

where $t(r,v)$ is the task length of task v if processed by resource r and $\tau_k(T)$ is k^{th} largest task in *task set T*.

Each task is assigned a deadline by propagating the order deadlines upwards in the TSG. For the last task of any order, its deadline is same as the order's deadline. The deadline $\delta(v_i)$ for a task v_i is computed from deadlines of its child tasks using Equation (2).

$$\delta(v_i) = \min_{v_c}\{\delta(v_c) - \omega(v_c) | v_c \in childs(v_i)\} \quad (2)$$

Before scheduling v_i, its slack time, $\sigma(v_i)$ is estimated from the completion time, Sp, of its parent nodes. The slack time is obtained using Equation (3).

$$\sigma(v_i) = \delta(v_i) - \max_{v_p}\{Sp(v_p) | v_p \in parents(v_i)\} - \omega(v_i) \quad (3)$$

The task set T is sorted on the basis of the σ-values of all its tasks. The priority $p(v_i)$ of task v_i with the lowest value of σ is assigned a value of 1 (i.e., the highest priority), the task with the next lower value is assigned a priority value of 2, and so on. We observed that the priority inversion between two tasks may lead to a better schedule with respect to our objective function. To provide a mechanism to control this flexibility and yet provide a goal-oriented schedule, we introduce the terms *priority inversion flexibility controller* (λ) and *effective priority* ($\rho(v_i)$), related by $\rho(v_i) = (p(v_i))^{\lambda} \times gene(i)$, where λ is a positive real number. It becomes gradually harder to invert priorities with increase in the value of λ. The tasks in T are sorted once again on the basis of their effective priorities. If two tasks are assigned the same resource, the conflict between them is resolved using their effective priority values. Note that a task is scheduled to start after: (i) its parent tasks have been processed; (ii) the resource assigned to that task becomes free.

4.4 Fitness Function

Initially we take a population size of $2m$ chromosomes, each having a distinct fitness value. For faster convergence towards a solution, we add a chromosome corresponding to a relaxed schedule in the first-generation population. The initial population is further enriched by adding a chromosome corresponding to a baseline earliest-due-date (EDD) schedule. (In EDD, earlier the due-date of an order is, higher is the priority assigned to the tasks from that order.) This chromosome guarantees that we get a result that is at least as good as the result obtained using EDD. The chromosome with a higher chance of becoming a solution should have a higher fitness value. Since our goal is to minimize the sum $\sum_{i=1}^{n}(\alpha_i \times s_i)$, we use the fitness value, $f = -(\sum_{i=1}^{n}(\alpha_i \times s_i))$.

4.5 Evolution Strategy

We use three evolution steps for generating next generation of candidates: *reproduction*, *crossover* and *mutation*. *Reproduction* is defined as the copying of good chromosomes from one generation to the other [8]. This ensures that we have candidates that are at least as good as what we had in the last generation. We copy top 30% of the total population with unique fitness values to the successive generation. *Crossover* is a technique to exchange genes between two chromosomes. We use the parameterized uniform crossover approach in this work [9]. The crossover procedure accounts for 50% of the chromosomes in the next generation. In *mutation*, the remaining part of the population is randomly created in the next generation. This step accounts for 20% of the chromosomes in the next generation.

4.6. Computational Complexity

The run-time complexity of the GA procedure depends on the number of generations that must be iterated through, before the convergence criterion is reached. We set a time-out of 15 minutes or an unaltered fitness value of best chromosome for 100 consecutive generations, whichever is earlier, as a convergence criterion for the heuristic to terminate. If the procedure runs for G generations, its time complexity is $O(Gm^3)$. The procedure for creating a schedule from a chromosome, shown in Figure 4, has $O(m^2)$ complexity. It is invoked for each chromosome of the population in each generation. Since the size of the population is $2m$, the overall heuristic is of $O(Gm^3)$ complexity.

5. Results and Comparison

The heuristic discussed above was implemented and applied to a small problem instance and then to real orders obtained from a PSP. We also implemented the baseline heuristics first-come first-served (FCFS) and EDD, and compared their results with the GA approach. In contrast to EDD, FCFS method assigns a higher priority to tasks belonging to orders having earlier accept-date. Resource allocation is carried out for the baseline cases using a round-robin strategy.

5.1 Simplified example

Our simplified example consists of fulfillment paths as shown in Figure 1. The examples of orders and the resource

Table 3: Table showing results for simplified example.

	Case	d_1	d_2	c_1	c_2
GA-based heuristic	(a)	loose	loose	637	607
	(b)	tight	tight	179	365
	(c)	loose	tight	398	350
	(d)	tight	loose	179	369
EDD	$d_1 > d_2$			320	350
	$d_1 < d_2$			197	365

Table 4: Results for real order.

n	m	Normalized value of objective function	G
3	120	0.92	538
6	230	0.85	703
10	386	0.78	753
15	602	0.71	697

library are taken from Table 1 and Table 2, respectively. This example serves as a micro-benchmark for our heuristic. As shown in Table 3, four different cases are created depending on how loose or tight the deadline of each order is. The effectiveness of our heuristic is reflected in the way it adapts itself to service the critical order before the order with a loose deadline. The parameters d_1, c_1, d_2 and c_2 are as defined in Section 2. Figure 5 shows a Gantt chart for schedules created in the cases (b) and (c). It can be seen that the tasks of the order O1 are scheduled when its urgency is detected, in contrast to the unhurried schedule created for the same order when it is not urgent.

The minimum possible value for $c_1(c_2)$ can be hand calculated; it was found to be 179(350) time units. The parameter c_1 attains its minimum value when O1 is critical. Similarly, c_2 achieves its minimum value when O2 is critical except for the case when both the orders are critical. This result can be attributed to resource contention due to limited number of resources.

It is worth noting that for EDD, even when $d_1 < d_2$, c_1 cannot not reach its minimum value. Being a multi-part product, the order O1 requires a higher priority for all tasks along its critical path, i.e., the path that requires most of the processing time, as shown in Figure 2. Since there is only one IP resource available in the resource pool, the two related tasks within O1 itself compete with one another for the lone resource. The simplistic nature of EDD prevents it from prioritizing tasks within the same order and hence EDD assigns equal priorities to all tasks within the same order. The GA-based heuristic does not suffer from this limitation; and it also optimizes resource allocation. Note that FCFS gives the same results as EDD for this example because sorting the two orders will always schedule O1 before O2, or vice versa.

5.2 Results for real orders

Real order data is obtained from a commercial digital PSP. Details about the order data or the fulfillment paths cannot be divulged due to confidentiality requirements. However, we provide sufficient details in order to convey the complexity of the optimization problem. Two fulfillment paths were considered; one with 20 processing stages and the other with 12 stages. In addition to this, each product has to be processed through 8 order-level stages. We set $\lambda = 1$ and $M = -1000$ and a time-out of order of 10 minutes. The time-out is determined according to the factory's order release scheme. Table 4 shows the total number of generations the heuristic ran before its termination (column 3) and the value of our objective function normalized with respect to that of EDD in column 4. A lower measure of this quantity indicates the effectiveness of our approach. The CPU time was only a few seconds for $m = 120$, a few minutes for $m = 230$ and $m = 386$, and in the order of tens of minutes when $m = 602$. As expected, the time spent per generation increases with an increase in the number of total tasks due to an increase in the population size and the chromosome size.

Figure 6 shows how the normalized value of our objective function varies with the number of generations. The graph is truncated at 400 generations because the subsequent generations registered negligible improvements. The heuristic appears to converge near $G = 300$. Setting this value of G for large problem instances such as for $n = 15$ will reduce the CPU time with acceptable results because near-optimal solutions are admissible as long as deadlines are met.

Table 4 demonstrates that the proposed heuristic method provides even better results with an increase in the number of tasks. With a larger number of tasks, the search space becomes bigger and hence presents a better opportunity to find a solution that is much better than the simplistic EDD method. As expected, the GA-based heuristic takes more CPU time compared to the other two baseline methods. The print factory releases the orders to the factory floor periodically with a fixed time interval. We set the upper limit of the computing time (order of 10 minutes) according to this fixed time interval. This ensures that our GA-based algorithm is able to always provide near-optimal solutions to the factory manufacturing execution software in time for the order release events.

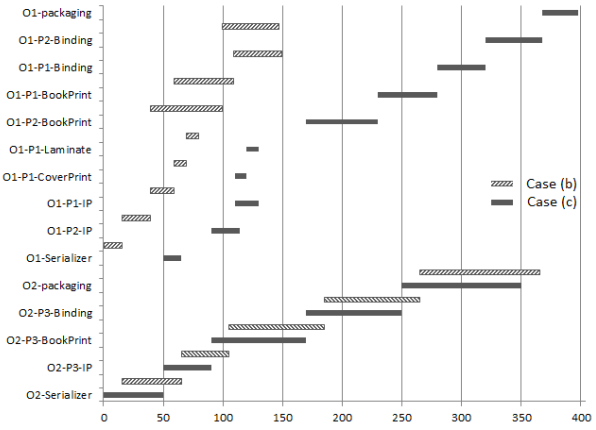


Figure 5: Gantt Chart showing schedule for orders in simplified example for case (b) and case (c).

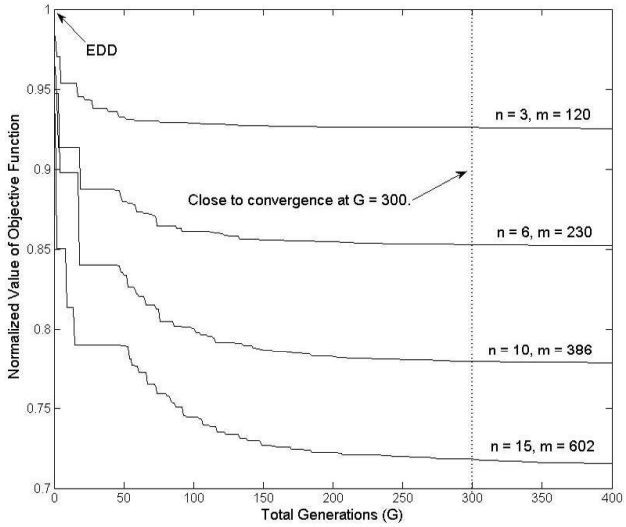


Figure 6: Performance of GA-based heuristic for real orders.

6. Conclusion and Future Work

We have demonstrated that a GA-based heuristic method can be used for simultaneous scheduling and resource binding in digital print optimization. The proposed method outperforms the baseline EDD and FCFS methods under varying deadline constraints. Even for a simple example illustrated in the paper, the baseline strategies were unable to match the quality of the results provided by the GA-based method. As partial schedules are progressively created for the orders, our GA heuristic takes into account the entire state of the system and is capable of assigning dynamic priorities based on the tasks that have already been scheduled and the tasks that are remaining in the system.

As part of ongoing work, we are modeling delays in print operations that could arise due to intermittent failures in printing devices and servers. We are also looking into planning issues that can be addressed by this optimization framework.

References

[1] Zeng, J., Jackson, S., Lin, I-Jong, Gustafson, M., Hoarau, E. and Mitchell, R., August 2011, "Operations Simulation of On-demand Digital Print", To appear in IEEE 13th Joint Int. Conf. Comp. Sci. Info. Tech., Congqing, China.

[2] Rai, S., Duke, C.B., Lowe, V., Quan-Trotter, C. and Scheermesser, T., 2009, "LDP Lean Document Production - OR enhanced productivity improvements for the print industry", Interfaces, 39(1), 69-90.

[3] Baker, K. R. and Scudder, G. D., 1990, "Sequencing with Earliness and Tardiness Penalties: A Review", Ops. Res. 38, 22-36.

[4] Hopp, W. and Spearman, M., 2007, "Factory physics (3rd Edition)", McGraw-Hill/Irwin.

[5] DeMicheli, G., 1994, "Synthesis and Optimization of Digital Circuits", McGraw-Hill, New York, NY

[6] Su, F. and Chakrabarty, K., 2004, "High level synthesis of digital microfluidics biochips", Proceedings of the IEEE International Conference on Computer-Aided Design, pages 223-228.

[7] Srinivas, M. and Patnaik, L. M., 1994, "Genetic algorithms: a survey", IEEE Comput. 27, 17-26.

[8] Bean, J. C., 1994, "Genetics and random keys for sequencing and optimization", ORSA J Comput 6, pages 154-160.

[9] Spears, W. M. and Dejong, K. A., 1991, "On the virtues of parameterized uniform crossover", In Proceedings of the International Conference on Genetic Algorithms, 230-236.

Reproduced with permission of the copyright owner. Further reproduction prohibited without permission.